Requirement Document: N8N Hands-On Beginner Course

Topic 4: JSON & Data Structure Basics

1. Document Control

Field	Value
Document Title	Requirement Document: Topic 4 - JSON & Data Structure Basics
Course	N8N - Hands On Beginner Course
Target Audience	Trainee / Employee
Version	1.0
Status	Approved
Last Updated	June 23, 2026

2. Objectives & Scope

2.1 Course Objective

The main objective of this learning module is to teach trainees and employees how data is structured and represented as JSON objects and arrays in n8n. By the end of this topic, the learner will know how to navigate nested JSON payloads, distinguish single-value fields from arrays in the data panel, and write expressions using dot and bracket notation to extract nested values.

2.2 Learning Scope

In Scope:

- Reading and expanding JSON structures inside the n8n execution data panel.
- Navigating the Table, JSON, and Schema views to analyze nested data structures.
- Identifying visual elements representing objects (`{}`) and arrays (`[]`).
- Writing n8n expressions using dot notation to access properties inside a nested JSON object (e.g., `{{ $json.customer.address.city }}`).

- Writing expressions using bracket index notation to access specific elements in a JSON array (e.g., `{{ $json.orders[0].total }}`).

Out of Scope:

- Advanced array manipulation using JavaScript array methods (e.g., `.map()`, `.filter()`, `.reduce()`).
- Using Loop or Split In Batches nodes to iterate over JSON lists.
- Writing or fixing malformed JSON strings outside the n8n UI environment.

3. Target Audience Requirements (Pre-requisites)

To successfully complete this hands-on exercise, trainees/employees must have:

- Completed Topic 1, Topic 2, and Topic 3.
- Basic understanding of key-value concepts.
- Access to an active n8n editor session with expression-building capability.

4. Functional Requirements

Requirement ID	Section	Requirement Description	Priority
REQ-001	Mock Data	The workflow shall generate a nested JSON payload containing a single-value string (`status`), a nested object (`customer`), and a multi-item array (`orders`).	**High**
REQ-002	Panel Interface	The trainee shall be able to inspect the structure of the generated payload, toggle views (JSON, Table, Schema), and expand/collapse nested attributes.	**High**
REQ-003	Single Value Access	The workflow shall include a data extraction node that accesses the nested `customer.name` field and maps it to a new flat field named `customerName`.	**High**
REQ-004	Nested Object Access	The workflow shall extract a nested property within a sub-object, specifically mapping `customer.address.city` to a top-level field `cityOfCustomer`.	**High**

Requirement ID	Section	Requirement Description	Priority
REQ-005	Array Index Access	The workflow shall extract a property from the first item of an array, specifically mapping `orders[0].total` to a top-level field `firstOrderTotal`.	**High**
REQ-006	Execution Output	The workflow execution shall successfully run and output a single JSON object containing exactly the three extracted fields: `customerName`, `cityOfCustomer`, and `firstOrderTotal`.	**High**

5. Technical Requirements

ID	Technical Requirement	Details
TR-001	Expression Resolution	The n8n expression engine must resolve nested dot-notation paths (`customer.address.city`) and array-indexing bracket-notation paths (`orders[0].total`) dynamically.
TR-002	Data Types Preservation	The extracted fields must preserve their native types: `customerName` (String), `cityOfCustomer` (String), and `firstOrderTotal` (Number).

6. Stakeholders and Personas

Persona	Role	Need
Trainee / Employee	Student	Needs to understand n8n's JSON data panel layout and learn expressions to map nested payloads between integration endpoints.
Instructor / Reviewer	Evaluation	Needs to confirm the student has written correct expressions for nested fields and array indices rather than hardcoding static values.

7. Assumptions & Constraints

- **Assumption 1 (ASM-001):** The trainee is using an n8n version that supports direct drag-and-drop mapping of nested values to auto-populate dot notation.
- **Constraint 1 (CON-001):** The array indexing in this topic is limited to static index retrieval (e.g. `[0]`) rather than dynamic variable indexing or looping.